Community Library Management System

Project Report

1.Introduction

Chosen Organization and Problem:

The tracking of stock, membership, and transaction lifecycle is a challenge faced by most small-scale libraries due to manual physical methods of record keeping. The lack of a single system brings out operational challenges, such as missing books, failure to return borrowed materials, and rule infractions such as overloading of borrowers 'capacity. The software evaluated in this report addresses such problems in local libraries through a lightweight and strong command line-based program.

purpose and Scope:

The main objective of this system is to automate the process of inventory control and lending based on strong library policies. The scope of this application includes:

- The creation of an electronic list of books, with consolidation of multiple entries of the same book in the inventory.
- Creating a list of community members with a strict limit on loaning.
- Managing the lending process along with 14-day automatic returns.
- Locally storing all the data without needing any external databases.

2.System Design

Main Classes and Relationships:

Library: Coordinator of the central system. It keeps in memory dictionaries of Books and Members as well as a sequentially sorted collection of Loan objects. It implements business rules and performs all data file transfers.

Book: Represents a particular title statistics. It stores total number of purchased copies as compared to available copies on shelves (available copies).

Member: Stores user demographic information as well as an internal list of book_ids borrowed by the Member.

Loan: Links a particular Member to a particular Book with time stamps of borrowing and returning that item.

Data Management and File Handling:

The data is stored in a structured form in flat csv files located in a dedicated data directory (files books.csv, members.csv, and loans.csv). The project fully decouples file handling from the user interface tier :

- Loading Data (load_all): At application launch time, the engine loads these files, reading line-by-line and calling class constructors (using the Python factory pattern approach from_row()). In case the files do not exist, it creates an empty repository.
- Saving Data (save_all): At exit time, by the menu selection, the engine serializes the in-memory states to text rows (to_row()), writing them to disk..

3.Implementation Overview

Core Python Concepts Applied:

- restriction & Factory Methods: The data attributes have been grouped nicely into descriptive entity classes. Using the classmethod decorator for from_row(cls, row) restricts the conversion from simple CSV string fields to object declarations.
- String Representations: Overridden str methods guarantee that any print function calls on library entities will return the nicely formatted status report.
- Exception Hierarchy: Instead of allowing general-purpose syntax errors raised by the system to bubble up to the top level, a custom base class LibraryError is defined in and its subclasses represent specific violations: BookNotFoundError, MemberNotFoundError, BookUnavailableError, and BorrowLimitError.
- Datetime Manipulation: The default Python datetime module provides date and timedelta objects that can easily handle constraint evaluation, adding L = 14 days to standard ISO format strings.

Readability and Modularity:

Modularity has been ensured due to proper separation of concerns. The interactive menu functionality is handled by “main.py”, the transactional constraints are taken care of by “library.py” and the data structures are provided within the models package (“book.py” ,”member.py” and ”loan.py”) and Readability has been ensured by using descriptive naming and documentation.

4.Testing and Demonstration

Example Inputs and Outputs

The user interface layer safely intercepts and reports system status adjustments leading to prevent unhandled failures:

```
==== Community Library ====
1. Add book
2. Register member
3. Borrow book
...
Choose an option (1-8): 3
Member ID: M002
book_id : 9780140328721
Done. Due back on 2026-07-09.
```

If a business rule is violated, the interface reports the exact exception message to the librarian:

```
Choose an option (1-8): 3
Member ID: M002
Book_id : 9780747532699
Could not lend book: No copies of 'Harry Potter' are available.
```

Verification and Correctness Testing:

- Boundary Testing: Confirmed that trying to check out the fourth book correctly raises a BorrowLimitError after satisfying the condition loans >= 3.
- Input Sanitization: Used try/except ValueError in numerical lists such as the number of books to avoid program crashes when the program takes input in non-numerical integer string form.
- Persistence Integrity Testing: the Data is verified by filling the library parameters, making a save-exit decision, and checking the resulting CSV text lines for matching cell data fields.

5.Reflection

Challenges and Lessons Learned:

Multi-entity relational mapping such as associating loans to members and books has been difficult in flat file formats like CSV. The most important realization was how helpful structure lookups (book_id) could be in associating multiple entities without the use of foreign keys in database relations. In addition, synchronization of counts had to be done in an atomic way (like decreasing availability statistics).

Future Improvements:

- Smart Search and Filters: Introduction of a partial match search and filters where lookup is done based on keywords for title and author in the search fields rather than exact matching of ID.
- Overdue Fines: Incorporation of fines in the Loan class based on day differences from the desired date of return.
- Interface: From the command line interface to GUI via desktop software such as tkinter or web portal.